

ENDPOINTS, PAYLOADS, FAILURE MODES

Technical reference

Everything needed to drive video creation from the LMS and to render the in-video checkpoint. **Base URL** `https://content-queen-production.up.railway.app`. Two separate APIs: an unauthenticated job API for creating a video, and a token-gated read API for consuming its checkpoint.

Supersedes the course-builder reference of September 2026.

The `/api/v1/courses/*` endpoints still answer, but nothing has been built on them and they are not the supported path. Build against the endpoints in this document. The unit of work is **one video**, created and approved individually.

1 • Authentication

Two different rules, because the two APIs carry different risk.

API	AUTH	WHY
<code>/api/v1/*</code>	Bearer token, required	Reads content metadata. Fails closed with <code>503</code> if the server has no token configured, rather than serving openly.
<code>/demo/make-video/*</code>	None; rate limited	Creation is behind an unlisted URL and capped at 6 videos/hour/IP. It spends money, so the cap is the control, not the secrecy.

Authorization: Bearer `<CONTENT_API_TOKEN>`

The token is a shared secret. Never put it in browser-side code – call
our API from your server, and serve the checkpoint to the player yourself.
Ask Aroma Tahir for the value; it is not printed in this document.

Check you are wired up correctly: `GET /api/v1`

Public, no token. Returns a machine-readable index of every endpoint with example URLs, and `"configured": true` when the server holds a token. If that says false, the read API will refuse everything and the problem is on our side.

2 · Creating a video — the job API

Four calls. Writing is free and fast; producing costs money and takes half an hour; publishing needs a human. Each is a separate request so each is a separate decision.

	ENDPOINT	RETURNS
POST	/demo/make-video	202 + {jobId, status} . Body: {"topic": "..."} . Starts research → script → gate.
GET	/demo/make-video/:jobId	Job state. Poll every 4–5s. Carries the script once written.
POST	/demo/make-video/:jobId/produce	202 . Spends money. Art, voice, render, QA. Body {"publish": true, "by": "name"} supplies the approval up front and skips the pause; omit it and the run stops at awaiting_review .
GET	/demo/make-video/:jobId/video	The finished MP4. Honours Range , so it scrubs in a player.
POST	/demo/make-video/:jobId/approve	202 . Publishes to YouTube. Body: {"by": "name"} .
GET	/demo/make-video/:jobId/script.md	The script as a Markdown file, with Content-Disposition set so a browser saves it. Every beat, plus the checkpoint with its answer and explanation.
GET	/demo/sample	The pinned demo topic, and the committed worked example if one has been captured.

```
POST /demo/make-video           // free, returns immediately
Content-Type: application/json
{ "topic": "how do I handle an angry parent on the phone" }

→ 202 { "jobId": "8fb136e2012c", "status": "running" }
```

The job is created and the request returns at once. Writing a gated script takes three to seven minutes, which is far too long to hold a connection open — an edge proxy will cut it. Poll instead.

3 · The job lifecycle

`status` is what to branch on. `stage` is for showing progress and will move backwards during a redraft, which is normal.

STATUS	MEANING	WHAT THE UI SHOULD DO
running	Researching, writing or gating.	Show <code>stage</code> . Keep polling.
written	Script done and passed the gate.	Render <code>script</code> . Offer the produce button.
producing	Art, voice, render, QA. A quality gate may send the script back for up to 2 rounds; the stage name moves backwards while it does, which is normal.	
awaiting_review	The video is finished. Not published.	Play <code>/video</code> . Offer the approve button.
publishing	Uploading to YouTube.	Keep polling.
published	Live, unlisted.	Show <code>produce.youtube.url</code> .
failed	The run stopped. <code>error</code> says where and why.	Show the error. The topic can be retried.

awaiting_review is a success state, not a failure

The video exists and passed QA. Treating it as an error is the most likely integration mistake — it is the state the pipeline is designed to end in, and only a person moves it on. `produce.status` mirrors it, and `produce.spendUsd` says what it cost.

6b · A fixed sample to build against

Identical in shape to a real checkpoint, so anything built against it works unchanged against a real video. It exists so the popup can be built and demonstrated without producing a video first.

```
GET /api/v1/videos/sample/checkpoints
Authorization: Bearer <CONTENT_API_TOKEN>
```

```
# The same envelope as any real video: timing{}, checkpoints[], the same
# guards. The question is a constant, so it is safe to screenshot, paste into a
# ticket, or keep as a test fixture.
```

Start here.

Build the whole popup — preamble, four options, right and wrong states, the explanation — against `sample`. Then change one string to a real `videoId` and nothing else needs to change.

4 · The script, once written

GET /demo/make-video/8fb136e2012c

```
{
  "jobId": "8fb136e2012c",
  "status": "written",
  "stage": "gate",
  "topic": "how do I handle an angry parent on the phone",
  "elapsedSeconds": 201,
  "error": null,
  "script": {
    "itemId": "made/how-do-i-handle-an-angry-parent-on-the-phone-nhl1",
    "slug": "how-do-i-handle-an-angry-parent-on-the-phone-nhl1",
    "title": "Handling an Angry Parent Phone Call",
    "interpretation": "You asked how to handle an angry parent on the phone; this
                      video narrows to the first ninety seconds of that call ...",
    "slo": "Given an angry parent on the phone, the teacher can ...",
    "scenario": "Ali is alone in the staff room at twenty to five ...",
    "gate": "READY",
    "redrafts": 1,
    "beats": [
      { "id": "01", "mode": "scene", "vo": "It is twenty to five, and ..." },
      { "id": "13", "mode": "info", "vo": "Confirmation is the hinge ..." },
      { "id": "14", "mode": "checkpoint", "vo": null },
      { "id": "15", "mode": "ali", "vo": "Reflect once, then explain." }
    ],
    "checkpoint": {
      "stem": "A parent opens the call shouting ...",
      "options": ["...", "...", "...", "..."],
      "answer": 1,
      "explain": "The tempting options all sound reasonable and all escalate ..."
    },
    "checkpointAfterBeat": "13"
  }
}
```

FIELD	TYPE	NOTES
interpretation	string	Show this. One sentence saying what the system narrowed the typed topic to, and why. It is how a user sees a wrong interpretation before paying to render it.
itemId	string	Opaque. Pass it back if you drive produce/approve from your own server.
beats[].mode	enum	<code>scene</code> · <code>ali</code> · <code>info</code> · <code>checkpoint</code> . The first three are drawn; a checkpoint never is.
beats[].vo	string null	Exactly one spoken sentence. null on a checkpoint beat — it is never spoken.
checkpoint	object	The question. <code>answer</code> is a 0-based index into <code>options</code> .
redrafts	integer	How many times the gate sent it back. Worth showing; it is a quality signal, not a fault.

5 · The finished video, awaiting approval

```
{
  "status": "awaiting_review",
  "produce": {
    "status": "awaiting_review",
    "stage": "review",
    "spendUsd": 0.558,
    "qa": {
      "factors": { "accuracy": 0.9, "objectives_coverage": 1, "post_production": 0.88,
        "visuals": 0.95, "storytelling": 0.95, "voiceover_quality": 0.9,
        "qa_at_each_step": 0.9 },
      "combined_score": 6.48,
      "weakest_factor": "post_production"
    },
    "elapsedSeconds": 1380
  }
}

# then, after POST .../approve
{ "status": "published",
  "produce": { "status": "done",
    "youtube": { "videoId": "2_IUH0sb6gY",
      "url": "https://youtu.be/2_IUH0sb6gY",
      "privacyStatus": "unlisted" } } }
```

Seven QA factors, each 0–1, combined out of 7. The publication threshold is 4.9; a video scoring below it does not reach this state at all. Store `combined_score` against the content record — it is the only quality number that exists.

6 · Reading a video's checkpoints

```
GET /api/v1/videos/:videoId/checkpoints
Authorization: Bearer <CONTENT_API_TOKEN>

{
  "videoId": "how-do-i-handle-an-angry-parent-on-the-phone-nhl1",
  "timing": { "trusted": true, "relativeTo": "...-nhl1_final.mp4",
    "introOffsetSeconds": 2.6, "introOffsetSource": "probed",
    "lessonSeconds": 137.3, "note": "atSeconds is measured from the
      start of the delivered file, which begins with the brand intro." },
  "checkpoints": [{
    "id": "q1",
    "beatId": "14",
    "atSeconds": 86.55,           // ← fire on this. Includes the intro.
    "lessonAtSeconds": 83.95,    // same point, intro excluded
    "preamble": "Let's answer a quick question, to check the concept landed.",
    "stem": "A parent opens the call shouting that you humiliated her daughter ...",
    "options": ["...", "...", "...", "..."],
    "correctIndex": 1,
    "explanation": "The tempting options all sound reasonable and all escalate ...",
    "explanationSource": "authored",
    "rendersInVideo": false,
    "pause": { "atBeatBoundary": true, "safe": true,
      "afterBeatId": "13", "beforeBeatId": "15",
      "boundaryLessonSeconds": 84.2, "leadSeconds": 0.25 }
  }]
}
```

7 · Five fields to respect

FIELD	RULE
atSeconds	Fire here. Measured from the start of the delivered file , brand intro included. Using <code>LessonAtSeconds</code> instead fires ~2.6s early.
timing.trusted	If false, do not fire. A beat length was not measured, so every time after it is a guess.
pause.safe	If false, do not fire. The checkpoint has no spoken sentence on one side, so there is no boundary to pause on.
rendersInVideo	Older videos (<code>true</code>) also draw the question on screen, so the popup duplicates it — you may prefer to let those play. New videos are always <code>false</code> .
explanationSource	<code>authored</code> = written for a learner who chose wrong. <code>video-note</code> = a short on-screen caption, thin as feedback. <code>missing</code> = show nothing but the correct answer.

Two guards, and both must pass.

`timing.trusted && pause.safe` . Either being false means the firing time is not dependable, and a popup landing mid-sentence is worse than no popup. Fail closed: play the video straight through.

8 · Listing what is available

```
GET /api/v1/videos          // every video with at least one checkpoint

{ "count": 20,
  "videos": [
    { "videoId": "autonomy-01-spectrum", "series": "autonomy",
      "path": "autonomy/autonomy-01-spectrum", "checkpoints": 1,
      "deliverableOnServer": false, "explanationsAuthored": 1,
      "timingTrusted": true, "questionStyle": "on-screen" }
  ] }
```

`questionStyle` is `"checkpoint"` for videos built the current way and `"on-screen"` for the older ones. `deliverableOnServer: false` is normal and expected — see overleaf.

9 · Firing the popup

```
// Fetch server-side; the token must never reach the browser.
const { timing, checkpoints } = await api.checkpointsFor(videoId);

// Fail closed. An untrusted time is worse than no popup.
const live = timing.trusted
  ? checkpoints.filter(c => c.pause.safe).sort((a, b) => a.atSeconds - b.atSeconds)
  : [];

let next = 0;
player.on('timeupdate', () => {
  const c = live[next];
  if (!c || player.currentTime < c.atSeconds) return;
  next++; // before showing, so a re-entrant tick cannot double-fire
  player.pause();
  showCheckpoint(c); // preamble, stem, options – your own UI
});

function onAnswer(c, chosen) {
  const right = chosen === c.correctIndex;
  record({ videoId, checkpointId: c.id, chosen, right }); // YOUR store
  if (right) return resumeAfterBrieflyConfirming();
  // Wrong: the explanation is the whole point. Show it with the correct
  // option marked, and let them continue – never block progress on it.
  showExplanation(c.explanation, c.options[c.correctIndex]);
}
```

- ❑ **Seeking.** A learner who scrubs past a checkpoint should not have it fire retroactively. Recompute `next` from `currentTime` on `seeked`.
- ❑ **Once per view.** Do not re-fire a checkpoint the learner has already answered in this sitting.
- ❑ **Never a dead end.** A wrong answer shows the explanation and continues. Gating progress on a formative check turns a teaching moment into an exam.

10 · What to store on your side

Our service holds no learner identity and no durable disk. The division is deliberate.

OURS	YOURS
The question, its options, the correct index, the explanation, and the second to fire at.	Who answered, what they chose, when, how many attempts, and everything built on that.
The YouTube link and the QA score at publication.	The content record those attach to, and the course a video belongs to.

```
POST /api/v1/videos/:videoId/checkpoints/:id/attempts

→ 501 Not Implemented
"The LMS is the system of record for answers; this service holds no learner identity."
```

This endpoint exists so that the answer to "where do attempts go?" is in the API itself rather than in a conversation. It returns 501 rather than accepting and discarding, which an earlier version did.

11 · Failure modes worth handling

Symptom	Cause and Response
503 on any /api/v1 route	The server has no token configured. It fails closed rather than serving openly. Ours to fix; check GET /api/v1 for configured.
404 on a videoId that worked	We keep no disk between releases. A video registered at runtime is gone after a deploy. Persist the checkpoint payload when you first read it.
deliverableOnServer: false	Expected, not an error. Video files are not committed to our repository. The YouTube link is the durable copy.
429 on make-video	Six videos per hour per IP. Creation spends real money.
status: failed mid-produce	A quality sensor rejected the work and the redraft budget ran out. The topic can be retried; the error names the stage and the finding.
Stuck in awaiting_review	Not stuck. It is waiting for a person, indefinitely and by design.

The single most important line in this document.

Our container has no persistent volume. **Read the checkpoint payload once and store it with your content record.** Do not call our API at playback time for every learner — cache it at publication, and treat our service as the source of truth at authoring time only.